OSS Ranking using d-set

OSS Researcher

2024-09-16

```r
# Load required packages.
library(openxlsx)   # Excel-related helpers
library(dplyr)      # Data manipulation
library(janitor)    # Clean column names
library(ggplot2)    # Plotting (loaded for downstream use)
library(readr)      # Read/write flat files
library(sos)        # Package/documentation search support
library(tidyverse)  # Tidy data workflow
library(lubridate)  # Date handling
library(tibble)     # rownames_to_column()
```

Overview

This R Markdown file prepares Bugzilla data, creates monthly, bi-monthly, and half-monthly maintenance datasets, computes d-set-based rankings, and evaluates rank stability using temporal windows and correlation measures.

1. Paths and Raw Data Import

```r
# Define input and output paths
input_path  <- "../data/"
output_path <- "../output/"

# Create directories if they do not exist
if (!dir.exists(input_path)) {
  dir.create(input_path, recursive = TRUE)
  message("Created 'data' folder at: ", normalizePath(input_path))
}

if (!dir.exists(output_path)) {
  dir.create(output_path, recursive = TRUE)
  message("Created 'output' folder at: ", normalizePath(output_path))
}

# Define input file
input_file <- file.path(input_path, "01-09-2023 to 31-08-2024.csv")

# Check if input file exists
if (!file.exists(input_file)) {
  stop(paste0(
    "Input file not found.\n",
    "Please place the file '01-09-2023 to 31-08-2024.csv' in the folder:\n",
```

```r
    normalizePath(input_path)
  ))
}

# Read the raw Bugzilla extract
init_dataset <- read.csv(input_file)

# Confirm successful read
cat("Input file loaded successfully. Rows:", nrow(init_dataset),
    "Columns:", ncol(init_dataset), "\n")
```

```
## Input file loaded successfully. Rows: 9139 Columns: 14
```

```r
 cat("\ninit_dataset gives the initial dataset fetched from BugZilla with some fieds not used in the stu
```

```
##
## init_dataset gives the initial dataset fetched from BugZilla with some fieds not used in the study
```

```r
View(init_dataset)
```

2. Basic Cleaning and Monthly Time Index Creation

```r
# Standardize column names and convert the update timestamp to Date format.
init_dataset <- clean_names(init_dataset)
init_dataset$updated <- as.Date(init_dataset$updated)

# Create a month-year label for monthly and bi-monthly aggregation.
init_dataset$month <- paste0(month(init_dataset$updated), "_", year(init_dataset$updated))

# Display the dimensions of the cleaned bug-level dataset.
cat("Dimensions of cleaned bug-level dataset (rows, columns):\n")
```

```
## Dimensions of cleaned bug-level dataset (rows, columns):
```

```r
dim(init_dataset)
```

```
## [1] 9139   15
```

```r
# Summarize monthly dataset coverage using distinct bug counts and date ranges.
month_dataset <- init_dataset %>%
  group_by(month) %>%
  summarise(
    n_b = n_distinct(bug_id),
    start_dt = min(updated),
    end_dt = max(updated),
    .groups = "drop"
  ) %>%
  arrange(start_dt)
cat("Dimensions of monthly summary dataset (rows, columns):\n")
```

```
## Dimensions of monthly summary dataset (rows, columns):
```

```r
# Display the dimensions of the monthly summary dataset.
dim(month_dataset)
```

```
## [1] 14  4
```

```r
cat("\n month_dataset gives the month-wise count for the unique bugs")
```

```
##
##  month_dataset gives the month-wise count for the unique bugs
```

```r
View(month_dataset)
```

3. Remove Incomplete Boundary Months

```r
# Exclude incomplete boundary months to maintain comparability.
month_data <- month_dataset %>%
  filter(month != "9_2024" & month != "8_2023")



cat("Dimensions of filtered monthly dataset after excluding boundary months (rows, columns):\n")
```

```
## Dimensions of filtered monthly dataset after excluding boundary months (rows, columns):
```

```r
dim(month_data)
```

```
## [1] 12  4
```

```r
#Number of months retained for analysis
n_month <- nrow(month_data)

cat("\n month_data gives the month-wise count for data under study, excluding edge months")
```

```
##
##  month_data gives the month-wise count for data under study, excluding edge months
```

```r
View(month_data)
```

4. Product-Status-Resolution Aggregation

There are 5 status under study - NEW, ASSIGNED, REOPENED are N-A-R used as-is in study

Other two status RESOLVED and VERIFIED are called F-Fixed in study as that belongs to resolution=FIXED in BugZilla

```r
# Count distinct bugs by month, product, status, and resolution.
n_prod <- length(unique(init_dataset$product))
cat("Number of distinct products in the dataset:\n")
```

```
## Number of distinct products in the dataset:
```

```r
n_prod
```

```
## [1] 16
```

```r
prod_dataset <- init_dataset %>%
  filter(month != "9_2024" & month != "8_2023") %>%
  select(month, product, status, resolution, bug_id) %>%
  group_by(month, product, status, resolution) %>%
  summarise(bugs = n_distinct(bug_id), .groups = "drop")
```

```r
cat("Number of aggregated records (month × product × status × resolution):\n")
```

```
## Number of aggregated records (month × product × status × resolution):
```

```r
nrow(prod_dataset)
```

```
## [1] 579
```

```r
cat("\n prod_dataset gives the final dataset at granular level with fields under study for main d-set")
```

```
##
##  prod_dataset gives the final dataset at granular level with fields under study for main d-set
```

```r
View(prod_dataset)
```

5. Construct Monthly N-F-A-R Matrices

5.1 New Bugs

```r
# Initialize product-by-month matrix for NEW bugs.
prods <- unique(prod_dataset$product)
mons  <- month_data$month

new_prod <- matrix(0, nrow = n_prod, ncol = n_month) %>% as.data.frame()
rownames(new_prod) <- prods
colnames(new_prod) <- mons

# Aggregate NEW bug counts.
new_prod_temp <- prod_dataset %>%
  filter(status == "NEW") %>%
  select(month, status, product, bugs) %>%
  group_by(month, product, status) %>%
  summarise(bugs = sum(bugs), .groups = "drop") %>%
  select(month, product, bugs) %>%
  as.data.frame()

# Populate the matrix.
n_r <- nrow(new_prod_temp)
for (i in 1:n_r) {
  q <- new_prod_temp[i, "month"]
  p <- new_prod_temp[i, "product"]
  new_prod[p, q] <- new_prod_temp[i, "bugs"]
}

View(new_prod)
cat("new_prod gives the count for each product and bi-monthly for status=NEW")
```

```
## new_prod gives the count for each product and bi-monthly for status=NEW
```

5.2 Fixed Bugs

```r
# Initialize product-by-month matrix for FIXED bugs.
fixed_prod <- matrix(0, nrow = n_prod, ncol = n_month) %>% as.data.frame()
rownames(fixed_prod) <- prods
colnames(fixed_prod) <- mons
```

```r
fixed_prod_temp <- prod_dataset %>%
  filter(resolution == "FIXED") %>%
  select(month, resolution, product, bugs) %>%
  group_by(month, product, resolution) %>%
  summarise(bugs = sum(bugs), .groups = "drop") %>%
  as.data.frame()

n_r <- nrow(fixed_prod_temp)
for (i in 1:n_r) {
  q <- fixed_prod_temp[i, "month"]
  p <- fixed_prod_temp[i, "product"]
  fixed_prod[p, q] <- fixed_prod_temp[i, "bugs"]
}

View(fixed_prod)
cat("fixed_prod gives the count for each product and bi-monthly for status=RESOLVED or VERIFIED or reso
```

```
## fixed_prod gives the count for each product and bi-monthly for status=RESOLVED or VERIFIED or resolu
```

5.3 Assigned Bugs

```r
# Initialize product-by-month matrix for ASSIGNED bugs.
assigned_prod <- matrix(0, nrow = n_prod, ncol = n_month) %>% as.data.frame()
rownames(assigned_prod) <- prods
colnames(assigned_prod) <- mons

assigned_prod_temp <- prod_dataset %>%
  filter(status == "ASSIGNED") %>%
  select(month, status, product, bugs) %>%
  group_by(month, product, status) %>%
  summarise(bugs = sum(bugs), .groups = "drop") %>%
  select(month, product, bugs) %>%
  as.data.frame()

n_r <- nrow(assigned_prod_temp)
for (i in 1:n_r) {
  q <- assigned_prod_temp[i, "month"]
  p <- assigned_prod_temp[i, "product"]
  assigned_prod[p, q] <- assigned_prod_temp[i, "bugs"]
}

 View(assigned_prod)
 cat("assigned_prod gives the count for each product and bi-monthly for status=ASSIGNED")
```

```
## assigned_prod gives the count for each product and bi-monthly for status=ASSIGNED
```

5.4 Reopened Bugs

```r
# Initialize product-by-month matrix for REOPENED bugs.
reopened_prod <- matrix(0, nrow = n_prod, ncol = n_month) %>% as.data.frame()
rownames(reopened_prod) <- prods
colnames(reopened_prod) <- mons
```

```r
reopened_prod_temp <- prod_dataset %>%
  filter(status == "REOPENED") %>%
  select(month, status, product, bugs) %>%
  group_by(month, product, status) %>%
  summarise(bugs = sum(bugs), .groups = "drop") %>%
  select(month, product, bugs) %>%
  as.data.frame()

n_r <- nrow(reopened_prod_temp)
for (i in 1:n_r) {
  q <- reopened_prod_temp[i, "month"]
  p <- reopened_prod_temp[i, "product"]
  reopened_prod[p, q] <- reopened_prod_temp[i, "bugs"]
}

View(reopened_prod)
 cat("reopened_prod gives the count for each product and bi-monthly for status=REOPENED")
```

```
## reopened_prod gives the count for each product and bi-monthly for status=REOPENED
```

6. Build the Bi-monthly N-F-A-R Dataset

```r
# Create a template bi-monthly dataset for all products.
n_m <- ncol(reopened_prod)
n_s <- 4
bimonthly_all <- matrix(0, nrow = n_prod * n_m / 2, ncol = n_s + 2) %>% as.data.frame()
colnames(bimonthly_all) <- c("Months", "Product", "New", "Fixed", "Assigned", "Reopened")

# Build bi-monthly labels by pairing adjacent months.
for (j in 1:n_m) {
  k <- (j + 1) %/% 2
  l <- 16 * (k - 1) + 1
  u <- 16 * (k - 1) + 16

  if (j %% 2 != 0) {
    bimonthly_all[c(l:u), 1] <- colnames(new_prod)[j]
  } else {
    bimonthly_all[c(l:u), 1] <- paste0(bimonthly_all[c(l:u), 1], " & ", colnames(new_prod)[j])
  }
}

# Populate product names for each bi-monthly block.
for (i in 1:n_prod) {
  bimonthly_all[i + n_prod * c(0, 1, 2, 3, 4, 5), 2] <- rownames(new_prod)[i]
}

# Convert monthly matrices to a long format, then aggregate to bi-monthly N-F-A-R values.
df_long <- bind_rows(
  as.data.frame(new_prod)      %>% rownames_to_column("Product") %>% mutate(Metric = "New"),
  as.data.frame(fixed_prod)    %>% rownames_to_column("Product") %>% mutate(Metric = "Fixed"),
  as.data.frame(assigned_prod) %>% rownames_to_column("Product") %>% mutate(Metric = "Assigned"),
  as.data.frame(reopened_prod) %>% rownames_to_column("Product") %>% mutate(Metric = "Reopened")
) %>%
```

```r
  pivot_longer(
    cols = -c(Product, Metric),
    names_to = "Month_Year",
    values_to = "Value"
  ) %>%
  pivot_wider(
    names_from = Metric,
    values_from = Value
  ) %>%
  arrange(Month_Year, tolower(Product))

# View(df_long)

month_levels <- colnames(new_prod)

# Create correctly ordered bi-monthly labels.
bi_labels <- sapply(seq(1, length(month_levels), by = 2), function(i) {
  if (i < length(month_levels)) {
    paste(month_levels[i], month_levels[i + 1], sep = " & ")
  } else {
    month_levels[i]
  }
})

# Aggregate adjacent months into bi-monthly N-F-A-R totals.
df_bi_sum <- df_long %>%
  mutate(
    Month_Year = factor(Month_Year, levels = month_levels),
    month_index = as.integer(Month_Year),
    group_id = (month_index - 1) %/% 2 + 1
  ) %>%
  group_by(Product, group_id) %>%
  summarise(
    New      = sum(New, na.rm = TRUE),
    Fixed    = sum(Fixed, na.rm = TRUE),
    Assigned = sum(Assigned, na.rm = TRUE),
    Reopened = sum(Reopened, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(Months = bi_labels[group_id]) %>%
  select(Months, Product, New, Fixed, Assigned, Reopened)

# View(df_bi_sum)

# Merge aggregated values into the final bi-monthly dataset.
bimonthly_all <- bimonthly_all %>%
  select(-New, -Fixed, -Assigned, -Reopened) %>%
  left_join(df_bi_sum, by = c("Months", "Product")) %>%
  arrange(Months, tolower(Product))

 cat("bimonthly_all gives the bi-monthly aggregated data for the main d-set demosntration")
```

```
## bimonthly_all gives the bi-monthly aggregated data for the main d-set demosntration
```

```r
 View(bimonthly_all)

write.csv(bimonthly_all, file = file.path(output_path, "bimonthly_all.csv"), row.names = FALSE)
```

7. Half-monthly Dataset Construction for each of the bi-monthly's indvidual d-set

```r
# Create half-month labels: a = days 1-15, b = days 16-end.
init_dataset$half_month <- paste0(
  month(init_dataset$updated), "_",
  ifelse(day(init_dataset$updated) <= 15, "a", "b"), "_",
  year(init_dataset$updated)
)
 cat("\n initiial dataset is used again to seggregate data with more granularity for each bi-monthly's i
```

```
##
##  initiial dataset is used again to seggregate data with more granularity for each bi-monthly's indivi
```

```r
dim(init_dataset)
```

```
## [1] 9139   16
```

```r
# Summarize half-month coverage.
half_month_dataset <- init_dataset %>%
  group_by(half_month) %>%
  summarise(
    n_b = n_distinct(bug_id),
    start_dt = min(updated),
    end_dt = max(updated),
    .groups = "drop"
  ) %>%
  arrange(start_dt)
cat("\nhalf_month_dataset gives the Half-monthly aggregated counts (4 for each of the bi-monthly), a dno
```

```
##
## half_month_dataset gives the Half-monthly aggregated counts (4 for each of the bi-monthly), a dnotes
```

```r
 View(half_month_dataset)
 cat("\n\ntotal sets for half-monthly(row*column)\n")
```

```
##
##
## total sets for half-monthly(row*column)
```

```r
dim(half_month_dataset)
```

```
## [1] 27  4
```

#trim-half-month-boundaries at the edges to discard from study

```r
# Remove incomplete half-month periods at both ends.
half_month_data <- half_month_dataset %>%
  filter(!half_month %in% c("9_a_2024", "9_b_2024", "8_a_2023", "8_b_2023"))


cat("\n\ntotal sets for half-monthly(row*column) under study by trimming egde cases\n")
```

```
##
##
## total sets for half-monthly(row*column) under study by trimming egde cases
```

```r
dim(half_month_data)
```

```
## [1] 24  4
```

```r
n_half_month <- nrow(half_month_data)
 cat("\nhalf_month_data gives the final sets for half-monthly calculations (4 for each of the bi-monthl
```

```
##
## half_month_data gives the final sets for half-monthly calculations (4 for each of the bi-monthly) un
```

```r
View(half_month_data)
write.csv(half_month_data, file = file.path(output_path, "half_month_data.csv"), row.names = FALSE)
```

```r
# Aggregate distinct bugs by half-month, product, status, and resolution.
n_prod <- length(unique(init_dataset$product))
#n_prod

half_prod_dataset <- init_dataset %>%
  filter(!half_month %in% c("9_a_2024", "9_b_2024", "8_a_2023", "8_b_2023")) %>%
  select(half_month, product, status, resolution, bug_id) %>%
  group_by(half_month, product, status, resolution) %>%
  summarise(bugs = n_distinct(bug_id), .groups = "drop")
 cat("\nhalf_prod_dataset gives the final dataset with product and status fields for half-monthly d-set
```

```
##
## half_prod_dataset gives the final dataset with product and status fields for half-monthly d-set calc
```

```r
View(half_prod_dataset)
 cat("\nnumber of half-monthly set under study must be <=16(products)*6(bi-monthly sets)*4(half monthly
```

```
##
## number of half-monthly set under study must be <=16(products)*6(bi-monthly sets)*4(half monthly with
```

```r
nrow(half_prod_dataset)
```

```
## [1] 972
```

```r
 cat("\nif the sets are less, there must be no status information for certain half-monthly-product-stat
```

```
##
## if the sets are less, there must be no status information for certain half-monthly-product-status gr
```

8. Build Half-monthly N-F-A-R Dataset and Windows R1-R6

```r
# Create half-monthly N-F-A-R counts by product.
half_monthly <- init_dataset %>%
  mutate(
    updated = as.Date(updated),
    half_month = paste0(
      month(updated), "_",
      ifelse(day(updated) <= 15, "a", "b"), "_",
      year(updated)
    )
  ) %>%
```

```r
  filter(
    !(year(updated) == 2024 & month(updated) == 9),
    !(year(updated) == 2023 & month(updated) == 8)
  ) %>%
  group_by(half_month, product) %>%
  summarise(
    New      = n_distinct(bug_id[status == "NEW"]),
    Fixed    = n_distinct(bug_id[resolution == "FIXED"]),
    Assigned = n_distinct(bug_id[status == "ASSIGNED"]),
    Reopened = n_distinct(bug_id[status == "REOPENED"]),
    .groups = "drop"
  ) %>%
  mutate(
    year_num  = as.integer(sub(".*_(\\d{4})$", "\\1", half_month)),
    month_num = as.integer(sub("_.*", "", half_month)),
    half_flag = ifelse(grepl("_a_", half_month), 1, 2)
  ) %>%
  arrange(year_num, month_num, half_flag, tolower(product)) %>%
  select(half_month, product, New, Fixed, Assigned, Reopened)

# Ensure all product-period combinations exist.
all_products <- sort(unique(init_dataset$product))
all_half_months <- sort(unique(init_dataset$half_month))

half_monthly <- half_monthly %>%
  complete(
    half_month = all_half_months,
    product = all_products,
    fill = list(New = 0, Fixed = 0, Assigned = 0, Reopened = 0)
  ) %>%
  filter(!half_month %in% c("9_a_2024", "9_b_2024", "8_a_2023", "8_b_2023"))

#length(unique(half_monthly$half_month))
#length(unique(half_monthly$product))
# View(half_monthly)

# Group four half-month periods into one temporal window (W1-W6).
half_monthly <- half_monthly %>%
  mutate(
    year_num  = as.integer(sub(".*_(\\d{4})$", "\\1", half_month)),
    month_num = as.integer(sub("_.*", "", half_month)),
    half_flag = ifelse(grepl("_a_", half_month), 1, 2)
  ) %>%
  arrange(year_num, month_num, half_flag, tolower(product)) %>%
  mutate(
    period_index = as.integer(factor(half_month, levels = unique(half_month))),
    W = paste0("R", ((period_index - 1) %/% 4) + 1)
  ) %>%
  select(W, half_month, product, New, Fixed, Assigned, Reopened)

colnames(half_monthly) <- c("W", "half_month", "Product", "New", "Fixed", "Assigned", "Reopened")
 cat("\nhalf_monthly gives the input dataset for temporal d-set study for each of the bi-monthly set\n")
```

```
##
```

```
## half_monthly gives the input dataset for temporal d-set study for each of the bi-monthly set
```

```r
 View(half_monthly)
write.csv(half_monthly, file = file.path(output_path, "half_monthly.csv"), row.names = FALSE)
```

9. d-set Calculations-common function

IVIF-set calculations and kappa are taken from baseline bi-monthly excel

```r
 cat("\nThis block has the d-set calculation steps and will be called for the calculation of d-set for m
```

```
##
## This block has the d-set calculation steps and will be called for the calculation of d-set for main a
```

```r
# Step I weights are computed externally in Excel.
# This function performs normalization, IVIF aggregation, and ranking for a given window.
minmax_norm_window <- function(data, window_name,
                               cost_cols = c("New", "Reopened"),
                               profit_cols = c("Fixed", "Assigned")) {

  # Min-max normalization for profit-type criteria.
  norm_fun_profit <- function(x) {
    x <- as.numeric(x)
    rng <- max(x, na.rm = TRUE) - min(x, na.rm = TRUE)
    if (is.na(rng) || rng == 0) return(rep(1, length(x)))
    (x - min(x, na.rm = TRUE)) / rng
  }

  # Min-max normalization for cost-type criteria.
  norm_fun_cost <- function(x) {
    x <- as.numeric(x)
    rng <- max(x, na.rm = TRUE) - min(x, na.rm = TRUE)
    if (is.na(rng) || rng == 0) return(rep(1, length(x)))
    (max(x, na.rm = TRUE) - x) / rng
  }

  # Restrict data to the requested window.
  out <- data %>% filter(W == window_name)
  if (nrow(out) == 0) {
    stop(paste("No rows found for window:", window_name))
  }

  # Normalize criteria.
  out <- out %>%
    mutate(across(all_of(profit_cols), norm_fun_profit)) %>%
    mutate(across(all_of(cost_cols), norm_fun_cost))

  # Aggregate min/max normalized values by product.

agg_norm <- out %>%
  group_by(Product) %>%
  summarise(
    # Original min/max
    New_min        = min(New, na.rm = TRUE),
    New_max        = max(New, na.rm = TRUE),
```

```r
    Fixed_min      = min(Fixed, na.rm = TRUE),
    Fixed_max      = max(Fixed, na.rm = TRUE),
    Assigned_min   = min(Assigned, na.rm = TRUE),
    Assigned_max   = max(Assigned, na.rm = TRUE),
    Reopened_min   = min(Reopened, na.rm = TRUE),
    Reopened_max   = max(Reopened, na.rm = TRUE),

    # 1 - value min/max
    New_1_min      = min(1 - New, na.rm = TRUE),
    New_1_max      = max(1 - New, na.rm = TRUE),
    Fixed_1_min    = min(1 - Fixed, na.rm = TRUE),
    Fixed_1_max    = max(1 - Fixed, na.rm = TRUE),
    Assigned_1_min = min(1 - Assigned, na.rm = TRUE),
    Assigned_1_max = max(1 - Assigned, na.rm = TRUE),
    Reopened_1_min = min(1 - Reopened, na.rm = TRUE),
    Reopened_1_max = max(1 - Reopened, na.rm = TRUE),

    .groups = "drop"
  )
# Construct Omega and Theta values.
agg_norm <- agg_norm %>%
  mutate(
    Omega_N_min = New_min / (New_max + New_1_max),
    Omega_N_max = New_max / (New_max + New_1_max),
    Theta_N_min = New_1_min / (New_max + New_1_max),
    Theta_N_max = New_1_max / (New_max + New_1_max),

    Omega_F_min = Fixed_min / (Fixed_max + Fixed_1_max),
    Omega_F_max = Fixed_max / (Fixed_max + Fixed_1_max),
    Theta_F_min = Fixed_1_min / (Fixed_max + Fixed_1_max),
    Theta_F_max = Fixed_1_max / (Fixed_max + Fixed_1_max),

    Omega_A_min = Assigned_min / (Assigned_max + Assigned_1_max),
    Omega_A_max = Assigned_max / (Assigned_max + Assigned_1_max),
    Theta_A_min = Assigned_1_min / (Assigned_max + Assigned_1_max),
    Theta_A_max = Assigned_1_max / (Assigned_max + Assigned_1_max),

    Omega_R_min = Reopened_min / (Reopened_max + Reopened_1_max),
    Omega_R_max = Reopened_max / (Reopened_max + Reopened_1_max),
    Theta_R_min = Reopened_1_min / (Reopened_max + Reopened_1_max),
    Theta_R_max = Reopened_1_max / (Reopened_max + Reopened_1_max)
  )

  # Apply alpha and beta interval weights from the external IVIF specification.
  agg_norm <- agg_norm %>%
    mutate(
      ABCD_N_min = Omega_N_min * 0.015,
      ABCD_N_max = Omega_N_max * 0.067,
      ABCD_N_min_beta = Theta_N_min * 0.074,
      ABCD_N_max_beta = Theta_N_max * 0.207,

      ABCD_F_min = Omega_F_min * 0.402,
      ABCD_F_max = Omega_F_max * 0.552,
```

```r
      ABCD_F_min_beta = Theta_F_min * 0.230,
      ABCD_F_max_beta = Theta_F_max * 0.379,

      ABCD_A_min = Omega_A_min * 0.455,
      ABCD_A_max = Omega_A_max * 0.788,
       ABCD_A_min_beta = Theta_A_min * 0.121,
      ABCD_A_max_beta = Theta_A_max * 0.242,

      ABCD_R_min = Omega_R_min * 0.00,
      ABCD_R_max = Omega_R_max * 0.083,
      ABCD_R_min_beta = Theta_R_min * 0.167,
      ABCD_R_max_beta = Theta_R_max * 0.417
    )

  # Compute aggregated IVIF measures.
  Kappa <- 2.1309
  agg_norm <- agg_norm %>%
    mutate(
      U_min = (ABCD_N_min + ABCD_F_min + ABCD_A_min + ABCD_R_min) / Kappa,
      U_max = (ABCD_N_max + ABCD_F_max + ABCD_A_max + ABCD_R_max) / Kappa,
      V_min = (ABCD_N_min_beta + ABCD_F_min_beta + ABCD_A_min_beta + ABCD_R_min_beta) / Kappa,
      V_max = (ABCD_N_max_beta + ABCD_F_max_beta + ABCD_A_max_beta + ABCD_R_max_beta) / Kappa
    )

  # Compute score bounds.
  agg_norm <- agg_norm %>%
    mutate(
      S_min = U_min - V_max,
      S_max = U_max - V_min
    )

  U_global <- abs(min(agg_norm$S_min, na.rm = TRUE))
  V_global <- max(agg_norm$S_max, na.rm = TRUE)

  # Compute final interval-valued d-set scores and ranks.
  agg_norm <- agg_norm %>%
    mutate(
      d_min = (S_min + U_global) / (U_global + V_global),
      d_max = (S_max + V_global) / (U_global + V_global)
    ) %>%
    arrange(desc(d_min), desc(d_max)) %>%
    mutate(Rank = row_number()) %>%
    arrange(tolower(Product))

  return(agg_norm)
}
```

```r
 cat("\nPlease check R1(to 6)_calc.csv in output directory to see step-wise d-set calculation resuls fo
```

```
##
## Please check R1(to 6)_calc.csv in output directory to see step-wise d-set calculation resuls for tem
```

```r
# Run the d-set calculations for R1-R6 and export the results.
R1_calc <- minmax_norm_window(half_monthly, "R1")
# View(R1_calc)
```

```r
write.csv(R1_calc, file = file.path(output_path, "R1_calc.csv"), row.names = FALSE)

R2_calc <- minmax_norm_window(half_monthly, "R2")
# View(R2_calc)
write.csv(R2_calc, file = file.path(output_path, "R2_calc.csv"), row.names = FALSE)

R3_calc <- minmax_norm_window(half_monthly, "R3")
# View(R3_calc)
write.csv(R3_calc, file = file.path(output_path, "R3_calc.csv"), row.names = FALSE)

R4_calc <- minmax_norm_window(half_monthly, "R4")
# View(R4_calc)
write.csv(R4_calc, file = file.path(output_path, "R4_calc.csv"), row.names = FALSE)

R5_calc <- minmax_norm_window(half_monthly, "R5")
# View(R5_calc)
write.csv(R5_calc, file = file.path(output_path, "R5_calc.csv"), row.names = FALSE)

R6_calc <- minmax_norm_window(half_monthly, "R6")
# View(R6_calc)
write.csv(R6_calc, file = file.path(output_path, "R6_calc.csv"), row.names = FALSE)

 cat("\nR1_calc gives the step wise d-set calculations for 1st bi-monthly ranks. Scroll far right for t
```

```
##
## R1_calc gives the step wise d-set calculations for 1st bi-monthly ranks. Scroll far right for the ra
```

```r
View(R1_calc)
```

```r
# Add a synthetic window label so the baseline bi-monthly dataset can be processed by the same function
add_window <- function(df, w_name) {
  df %>% mutate(W = w_name) %>% relocate(W, .before = 1)
}

# View(bimonthly_all)
nm <- "R_star"
R_star_calc <- minmax_norm_window(add_window(bimonthly_all, nm), nm)

View(R_star_calc)
 cat("\nR_star_calc gives the step wise d-set calculations for aggregated bi-monthly d-set-the main R*.
```

```
##
## R_star_calc gives the step wise d-set calculations for aggregated bi-monthly d-set-the main R*.
```

```r
write.csv(R_star_calc, file = file.path(output_path, "R_star_calc.csv"), row.names = FALSE)
```

10. Rank Comparison and Correlation Analysis

```r
# Build a combined rank table containing the baseline and all temporal rankings.
rank_compare <- data.frame(
  Product = sort(unique(bimonthly_all$Product)),
  stringsAsFactors = FALSE
)

# Helper function to extract product-rank pairs with a new rank column name.
```

```r
get_rank_df <- function(df, rank_col_name) {
  df %>%
    select(Product, Rank) %>%
    rename(!!rank_col_name := Rank)
}

rank_compare <- rank_compare %>%
  left_join(get_rank_df(R_star_calc, "R_star"), by = "Product") %>%
  left_join(get_rank_df(R1_calc, "R1"), by = "Product") %>%
  left_join(get_rank_df(R2_calc, "R2"), by = "Product") %>%
  left_join(get_rank_df(R3_calc, "R3"), by = "Product") %>%
  left_join(get_rank_df(R4_calc, "R4"), by = "Product") %>%
  left_join(get_rank_df(R5_calc, "R5"), by = "Product") %>%
  left_join(get_rank_df(R6_calc, "R6"), by = "Product") %>%
  arrange(tolower(Product))
cat("\nrank_compare gives the ranks R* and temporal windows ranks R1-R6.\n")
```

```
##
## rank_compare gives the ranks R* and temporal windows ranks R1-R6.
```

```r
View(rank_compare)
write.csv(rank_compare, file = file.path(output_path, "rank_compare.csv"), row.names = FALSE)
```

```r
# Define comparison pairs against the baseline and between adjacent temporal windows.
pairs_vs_rstar <- data.frame(
  Pair = c("R1 vs R_star", "R2 vs R_star", "R3 vs R_star", "R4 vs R_star", "R5 vs R_star", "R6 vs R_sta
  X = c("R1", "R2", "R3", "R4", "R5", "R6"),
  Y = c("R_star", "R_star", "R_star", "R_star", "R_star", "R_star"),
  stringsAsFactors = FALSE
)

pairs_adjacent <- data.frame(
  Pair = c("R1 vs R2", "R2 vs R3", "R3 vs R4", "R4 vs R5", "R5 vs R6"),
  X = c("R1", "R2", "R3", "R4", "R5"),
  Y = c("R2", "R3", "R4", "R5", "R6"),
  stringsAsFactors = FALSE
)

corr_pairs <- bind_rows(pairs_vs_rstar, pairs_adjacent)

# Compute Spearman and Kendall correlations, along with p-values.
corr_results <- corr_pairs %>%
  rowwise() %>%
  mutate(
    Spearman = cor(rank_compare[[X]], rank_compare[[Y]], method = "spearman", use = "complete.obs"),
    Kendall = cor(rank_compare[[X]], rank_compare[[Y]], method = "kendall", use = "complete.obs"),
    Spearman_p = suppressWarnings(cor.test(rank_compare[[X]], rank_compare[[Y]], method = "spearman", e
    Kendall_p = suppressWarnings(cor.test(rank_compare[[X]], rank_compare[[Y]], method = "kendall", exa
  ) %>%
  ungroup() %>%
  select(Pair, Spearman, Spearman_p, Kendall, Kendall_p)
cat("\ncorr_results gives pairwise Spearman correlation and Kendall's tau.\n")
```

```
##
## corr_results gives pairwise Spearman correlation and Kendall's tau.
```

```r
View(corr_results)
write.csv(corr_results, file = file.path(output_path, "rank_correlations.csv"), row.names = FALSE)
```

Time taken in running the code

```r
end_time <- Sys.time()

time_taken <- difftime(end_time, start_time, units = "secs")

timing_summary <- data.frame(
  start_time = start_time,
  end_time = end_time,
  execution_time_sec = as.numeric(time_taken)
)

print(timing_summary)
```

```
##            start_time            end_time execution_time_sec
## 1 2026-04-03 20:48:32 2026-04-03 20:48:36           4.166309
```